Comprendre les évolutions du projet LoRa APRS

Fonctionnement du code, apports pour les opérateurs et limites actuelles

Version française du 15 octobre 2026. Référence du firmware iGate : branche `rxt`, commit `2cebfe6` du 13 octobre 2026. Graywolf est une application distincte, examinée dans le dépôt associé `graywolf-rxt`.

1. Ce qui évolue par rapport au firmware CA2RXU

Le firmware de Ricardo Guzman, CA2RXU, fournit déjà la communication LoRa, le digipeater APRS, la passerelle APRS-IS, les interfaces TNC et la WebUI. Le projet conserve cette base et prolonge son fonctionnement dans deux directions : mieux décrire ce qui se passe sur la radio et mieux conserver les informations lors des échanges avec les applications.

Un paquet APRS dit qui émet, quel chemin il demande et quel contenu il transporte. Son chemin peut ensuite montrer les relais utilisés. Il ne dit cependant pas, à lui seul, avec quel niveau chaque relais a reçu le paquet, ni combien de temps ce relais a attendu avant de le retransmettre. RXT, dont le principe vient de Jon Adams, N7UV, ajoute ces observations. Le flux JSON permet ensuite de les exploiter avec le paquet dans Graywolf.

Le travail porte aussi sur le comportement du relais : gestion des chemins, limitation des répétitions, apprentissage d’un chemin de retour pour les messages et réponses locales. Un transport radio binaire expérimental s’y ajoute pour réduire la taille de certains paquets tout en conservant leur contenu exact.

Ces fonctions se répartissent entre plusieurs logiciels. L’iGate mesure, relaie et exporte. Graywolf conserve les réceptions, décode le contenu APRS lorsqu’il le reconnaît et présente les stations et les liaisons. Le chemin TNC2/JSON conserve les identités étendues, comme `F4MLV-MC` ou `F4JJE-16`. Le validateur actuel de l’iGate accepte aussi ces suffixes pour APRS-IS et le relais, sous réserve des autres règles de traitement. La limite concerne leur conversion en AX.25 sur l’interface KISS : ces suffixes ne peuvent pas y être représentés fidèlement.

2. Le paquet, son identité et ses interfaces

Le texte TNC2 comme représentation du paquet

Voici un exemple construit pour la lecture :

```
F4MLV-7>APRS,WIDE1-1:>Bonjour LoRa
```

`F4MLV-7` est la source. `APRS` est la destination du paquet. `WIDE1-1` demande un relais selon les règles de cet alias. Le deux-points termine l'en-tête ; le champ d'information commence ici par `>`, qui désigne un statut.

Après le passage par un digi, le chemin change pour indiquer le relais utilisé. Cette modification du chemin doit rester distincte d'une modification du message lui-même. Le destinataire doit pouvoir retrouver le contenu APRS initial, auquel les mesures de réception apportent un contexte supplémentaire.

L'en-tête TNC2 est textuel, mais le champ d'information peut contenir des octets binaires, notamment avec Mic-E. Un affichage lisible n'est donc pas toujours une copie complète des données reçues. C'est la raison de la copie en Base64 dans le flux JSON : elle permet de transporter les octets sans les perdre dans une conversion de caractères.

Pourquoi conserver les suffixes exacts

Un modèle textuel peut conserver `F4JJE-16` ou `NN7LE-GS` tels qu'ils ont été reçus. Les convertir immédiatement en une adresse AX.25 classique poserait un problème : le SSID AX.25 n'offre que les valeurs 0 à 15 et ne représente pas un suffixe alphabétique. Une conversion permissive pourrait alors changer l'identité de la station.

Graywolf conserve l'adresse textuelle d'origine et vérifie sa représentation exacte lorsqu'une sortie exige AX.25. Une identité non représentable est refusée à cette conversion. Elle peut néanmoins rester dans le journal, le modèle TNC2 et les données JSON.

À cette frontière AX.25, Graywolf refuse une identité qu'il ne peut pas représenter exactement. L'encodeur KISS historique de l'iGate utilise une conversion numérique du suffixe et ne réalise pas le même contrôle strict : il peut donc altérer l'identité. Le chemin TNC2 direct conserve les identités étendues entre Graywolf et la radio sans cette conversion. Le validateur textuel de l'iGate accepte les suffixes alphanumériques d'un ou deux caractères, dont `-MC`, `-GS` et `-16` ; les autres règles d'admission d'APRS-IS et du relais continuent de s'appliquer.

Trois interfaces à distinguer

Le **TNC2 direct** échange des enregistrements texte entre l'application et le TNC, par TCP ou série. **KISS** transporte des trames AX.25 binaires pour les logiciels et matériels qui en ont besoin. Le **flux JSON** décrit les réceptions et leurs mesures à destination d'une application.

Le TNC2 direct évite un aller-retour par AX.25 lorsque les deux extrémités savent déjà traiter le texte. Dans l'iGate, le réglage `tnc.kissProtocol` choisit le mode KISS ou TNC2 pour les interfaces série et TCP. Dans Graywolf, le TNC2 direct sert aussi à émettre des messages, leurs reprises et des balises sur un canal explicitement autorisé. Recevoir du JSON ou ouvrir une connexion de réception n'autorise pas à lui seul une émission.

3. Comment RXT rend les réceptions observables

Une mesure appartient au récepteur qui l'a faite

Prenons le chemin historique décrit par Jon : CAVECR émet, SOMTNX relaie, puis TSRXBX reçoit. Le RSSI mesuré par SOMTNX décrit la liaison de CAVECR vers SOMTNX. Le RSSI local de TSRXBX décrit la liaison suivante. Ces deux observations concernent des récepteurs et des sauts différents.

SOMTNX peut transporter sa mesure avec le paquet sans disposer d'Internet. Le récepteur final récupère ainsi les observations des relais précédents et y ajoute sa propre réception locale lors de l'export. Graywolf peut alors montrer des liaisons dirigées, plutôt qu'une qualité unique attribuée à tout le trajet.

RXT transporte quatre valeurs par relais : le niveau reçu RSSI en dBm, le rapport signal sur bruit SNR en dB, l'écart de fréquence FO en Hz et le délai de relais TTH en millisecondes. Le TTH va de la fin de réception à la fin d'émission du relais. Dans le code, il comprend l'attente en file, l'attente d'accès au canal et le temps d'antenne calculé pour la trame transmise. Il ne mesure donc pas uniquement le temps de traitement du processeur.

Les mesures sont quantifiées pour tenir dans un petit nombre de caractères. Le code actuel représente le SNR de -24 à +20 dB par pas de 0,5 dB. Le codage du TTH dépend du profil LoRa ; sa lecture exige des paramètres compatibles. Une valeur historique plafonnée à +12 dB ne doit pas être relue comme une mesure précise issue de la nouvelle échelle.

Base-89 pour RXT, Base64 pour JSON

Le code nomme le codage compact RXT **Base-89**. Chaque mesure devient un indice, puis un caractère ASCII obtenu en ajoutant 33 à cet indice. Le tuple n'est pas converti en bloc comme le serait une chaîne Base64 : chacun de ses caractères représente un champ distinct. Par exemple, un SNR de 0 dB donne l'indice 48 sur l'échelle actuelle, donc le caractère ASCII 81, `Q`.

Le nom Base-89 demande une précision pour cette implémentation. L'identifiant v2 utilise exactement 89 valeurs, par le calcul `33 + (empreinte modulo 89)`, soit les caractères `!` à `y`. Le SNR utilise lui aussi les indices 0 à 88. En revanche, les encodeurs RSSI, FO et TTH autorisent les indices 0 à 89 inclus, soit jusqu'au caractère `z` et donc 90 valeurs possibles. Le code actuel n'applique ainsi pas un alphabet uniforme de 89 symboles à tous les champs, malgré son nom.

Le **Base64 du JSON** a un autre rôle : transporter sans perte les octets du paquet APRS dans un document JSON, y compris les données binaires Mic-E. Il ne désigne pas le codage des mesures RXT sur la radio. Enfin, la **télémétrie APRS encodée de station** utilise des champs en **Base91**, notamment pour ses taux d'activité. Ces trois codages concernent des données et des interfaces différentes.

Références du codage RXT : constantes et fonctions d'encodage de `src/lora_utils.cpp`, fonction `fingerprint` de `src/rxt_protocol.cpp`.

Attribuer les mesures au bon relais

Dans la version texte RXT v2, un groupe de cinq caractères contient un identifiant puis les quatre mesures. L'identifiant est une empreinte calculée à partir de l'indicatif du relais, suffixe compris et sans l'étoile du chemin. Le décodeur confronte cette empreinte aux noms des relais utilisés.

Cette méthode supprime la dépendance à une liste blanche locale pour attribuer les tuples v2. Elle permet aussi de contrôler qu'un suffixe ressemblant à du RXT correspond au chemin. Elle garde une limite : une empreinte sur 89 valeurs peut être commune à plusieurs noms. L'ordre du chemin résout certains cas ; si plusieurs propriétaires restent possibles, le décodeur signale l'ambiguïté au lieu de choisir arbitrairement.

Un relais sans RXT peut figurer dans le chemin sans fournir de mesures. L'absence de données ne signifie pas que le saut n'a pas eu lieu. Le firmware conserve au plus trois tuples et n'ajoute pas de RXT aux messages APRS, ACK ou REJ, même lorsqu'ils sont encapsulés dans une trame tierce.

Préserver le texte APRS en mode radio texte

Les premières représentations ajoutaient un suffixe RXT visible au paquet. Le transport texte actuel place le RXT dans un bloc après un octet nul : le paquet APRS précède cet octet, puis viennent un marqueur et les mesures.

Un firmware qui lit la réception comme une chaîne C s'arrête à l'octet nul. Il retrouve donc le paquet APRS sans les mesures ajoutées. Ce comportement évite que cet iGate tiers transmette le RXT comme une partie du contenu utilisateur sur APRS-IS. En contrepartie, un relais classique qui retransmet seulement le texte perd les mesures cachées des sauts précédents.

Cette compatibilité dépend du comportement du lecteur tiers. Elle ne garantit pas tous les firmwares possibles. Le code continue aussi à lire certains trailers visibles reconnus ; dans cette représentation texte interne, un commentaire ressemblant par hasard à un trailer valide conserve un risque de confusion.

4. Du récepteur à Graywolf

Le contexte accompagne chaque paquet

À la réception, l'iGate capture les mesures locales avant qu'une nouvelle opération radio ne les change. Si le digi décide de relayer, le paquet et son contexte de réception entrent ensemble dans la file d'émission. Un paquet retardé doit garder ses propres mesures même si un autre paquet est reçu entre-temps.

La boucle principale distribue ensuite la réception aux fonctions activées : APRS-IS, digi, JSON, interfaces TNC, MQTT et journal SD selon leurs conditions respectives. Les sorties applicatives utilisent le paquet nettoyé du RXT reconnu. Les mesures restent accessibles séparément dans les sorties qui savent les décrire.

Le bouton d'activation RXT commande l'ajout de la mesure du relais lors d'une retransmission admissible. Le désactiver ne coupe pas la réception RXT, son décodage, son affichage ni le nettoyage avant APRS-IS.

Le JSON conserve le paquet et ses observations

Le flux `/api/v1/aprs/stream` fournit un objet JSON par ligne. Un événement de réception associe les octets TNC2 en Base64, les mesures locales, les paramètres radio et les sauts RXT décodés. Le texte lisible est une projection optionnelle, émise seulement lorsque les caractères peuvent être représentés sans ambiguïté dans cette forme.

En fonctionnement normal, hors ecoMode, le circuit JSON peut aussi conserver une réception non vide avec CRC valide que les fonctions APRS refusent ensuite, par exemple une enveloppe mal formée ou une source en liste noire. Observer une réception n'équivaut donc pas à l'autoriser sur APRS-IS ou à la répéter sur la radio.

L'iGate garde dix événements en mémoire et fournit un curseur pour reprendre après une déconnexion. Si le curseur demande une histoire devenue indisponible, un événement `gap` le signale. Cette mémoire permet une reprise courte ; elle ne remplace pas l'archivage dans l'application.

Ce que l’opérateur peut lire dans Graywolf

Graywolf conserve les réceptions, décode le contenu APRS qu’il reconnaît et alimente le journal et la carte. Les mesures RXT décrivent les liaisons distantes. La réception locale du dernier appareil décrit la liaison depuis le dernier relais utilisé, ou depuis la source en réception directe. Aucun TTH local n’est inventé.

Une liaison n’est dessinée que si les positions nécessaires sont connues. L’absence d’un trait sur la carte peut donc venir d’une position inconnue, même si les mesures existent. Les couleurs d’âge des liaisons ne constituent pas une échelle de qualité radio. Pour diagnostiquer un saut, il faut lire ses mesures et son horodatage.

5. Ce qui change dans les relais et les messages

Le chemin reste une décision de routage

Le digi vérifie le chemin et son mode avant de mettre un paquet en file. Les évolutions comprennent les alias régionaux configurables, leur nombre de sauts borné, les chemins explicites et une garde contre le retour par son propre relais. Une fenêtre locale de 25 secondes limite les répétitions d’un même paquet et sépare la consommation pour le relais de celle utilisée pour apprendre une route.

L’ajout de RXT reste subordonné à la place disponible. S’il ne tient plus, le firmware peut relayer le paquet sans nouveau tuple. Si le paquet APRS lui-même dépasse la limite admise, il refuse l’émission sans tronquer le contenu. Une mise en file ne constitue toutefois ni une preuve d’émission RF ni une preuve de livraison.

Le retour d’un message suit une route apprise

Lorsqu’une station est réellement entendue par LoRa, l’iGate peut apprendre les relais qui ont livré son paquet. Si le chemin utilisé comprend A puis B, le chemin retour devient B puis A. Une réception directe enseigne une route directe.

Un message reçu ensuite d’APRS-IS pour cette station récemment entendue utilise cette route, sous réserve des réglages de passerelle. La première copie retenue dans la fenêtre locale fournit le chemin. Cette politique favorise la première arrivée ; elle ne prétend pas sélectionner le meilleur signal. La route expire et disparaît au redémarrage. Sans route valable, la descente APRS-IS n’utilise pas le chemin général de balise comme repli.

Ce mécanisme traite le retour RF pour un iGate donné. Il ne choisit pas un émetteur unique lorsque plusieurs iGates peuvent joindre la même station. Cette coordination reste un autre problème.

Acquitter une copie sans répéter son traitement

Si un ACK est perdu, une copie suivante du message peut légitimement demander un nouvel ACK. Les corrections locales distinguent cet acquittement de l'exécution de la requête : elles permettent de renvoyer un ACK sans produire une seconde réponse dans la fenêtre prévue. Elles conservent aussi le reply-ack et font répondre l'identité effectivement adressée.

Les réponses RF locales peuvent employer leur chemin de repli quand aucune route utilisable n'est connue. Cette règle concerne un autre circuit que la descente des messages APRS-IS décrite ci-dessus.

Le contrôle distant ajoute une vérification distincte : une commande doit correspondre au contrôleur configuré et porter une signature HMAC liée à sa cible, à son compteur et à son contenu. Un état persistant des compteurs protège contre la réexécution. Cela authentifie ces commandes sans chiffrer leur contenu ; le trafic APRS ordinaire n'acquiert pas cette authentification.

6. La télémétrie de station complète RXT

RXT répond à une question sur une réception particulière. La télémétrie APRS de station décrit des tensions et une activité agrégée. Les taux sont calculés à partir des compteurs et du temps réellement écoulé entre rapports, puis exprimés en paquets par heure.

`RX_rate` compte les réceptions acceptées après filtrage de la liste noire. `RelRate` compte les paquets acceptés et mis en file par le digi. `DrpRate` compte des rejets selon les règles applicatives. Ils ne forment pas un bilan de livraison : un paquet reçu par un iGate dont le relais est désactivé peut augmenter RX_rate sans produire ni relais ni rejet du digi. Un paquet en liste noire peut augmenter DrpRate sans augmenter RX_rate.

Ces indicateurs servent à lire l'activité du site, avec leurs noms et unités. Ils ne permettent pas à eux seuls de calculer les pertes radio ou le succès des livraisons. L'origine du pic de taux signalé par Jon reste hors du présent document.

7. Le format radio natif expérimental

Le TNC2 direct entre logiciels et le format natif sur la radio sont deux mécanismes indépendants. Le premier reste textuel. Le second remplace l'enveloppe radio texte par un codage binaire compact qui doit permettre de reconstruire le paquet TNC2 exact.

Le codec compacte notamment les adresses et les positions prises en charge. Il peut coder le texte restant en Huffman si cela réduit sa taille. Il vérifie son propre résultat en le décodant. Les informations sans codage spécialisé peuvent rester brutes dans l'enveloppe native ; si la trame ne peut pas être encodée, l'émission se replie en texte.

Le réglage d'émission propose le texte par défaut, le natif, ou le texte suivi du natif pour les essais. Ce dernier mode produit deux copies. Les récepteurs doivent comprendre le format choisi ; un appareil sans codec natif ne bénéficie pas automatiquement de cette coexistence.

Le RXT natif rattache les mesures à des positions précises du chemin. Il évite ainsi l'empreinte texte pour cette association. Depuis le correctif du 13 octobre, ce contexte indexé reste séparé du texte à la réception, en file et lors du relais. Quand le chemin change, les mesures suivent les indicatifs complets. Une sortie ultérieure en texte revient cependant aux limites d'attribution par empreinte.

L'intérêt recherché est une réduction de la taille et du temps d'antenne lorsque le paquet s'y prête. Ce transport n'augmente pas à lui seul la sensibilité du récepteur ou la portée du site. Son format reste expérimental.

8. Ce qui a été vérifié et où lire le code

Les comptes rendus de septembre documentent le parcours Mic-E et les identités étendues vers Graywolf, ainsi que des échanges de messages. L'essai complémentaire du 28 septembre établit une livraison à un terminal par deux relais physiques, avec retour de l'ACK directement à l'iGate. Son apprentissage de route utilise un paquet synthétique : ce détail borne la conclusion.

Les essais du 13 octobre vérifient les octets effectivement retransmis sur RF. Ils montrent que l'association RXT native survit à une réécriture du chemin et à une attente en file, y compris avec deux noms partageant la même empreinte texte. Ils ne démontrent ni une portée de site ni tous les chemins et profils possibles.

Pour suivre le fonctionnement dans les sources, le point d'entrée est la boucle de `src/LoRa_APRS_iGate.cpp`. `src/lora_utils.cpp` lit la radio, capture le contexte et prépare l'émission. `src/digi_utils.cpp` décide du relais ; `src/station_utils.cpp` gère notamment la file et les routes apprises. `src/rxt_protocol.cpp` traite le RXT texte et `src/native_aprs.cpp` le codec natif. Les sorties JSON et TNC sont dans les modules `aprs_json_utils` et `tnc_utils`.

Les références techniques et les preuves matérielles sont séparées :

- `docs/RXT.md` : mesures, transport, interfaces et télémétrie de station.
- `docs/RXT_V2_fr.md` : motivation et attribution des tuples texte v2, à lire avec le transport actuel décrit dans RXT.md.
- `docs/REVERSE_MESSAGE_ROUTING_fr.md` : apprentissage et utilisation du chemin retour.

- `docs/VALIDATION_CORRECTIFS_RF_20260928_fr.md` : essais des chemins, réponses et messages.
- `docs/LORA_APRS_NATIVE_FRAME_DRAFT0_fr.md` et `docs/VALIDATION_RXT_NATIVE_RF_20261013_fr.md` : codec et essais RXT natifs.
- Dans Graywolf, `docs/wiki/rxt-telemetry.md` et `docs/wiki/lora-tnc2-transceiver.md` : réception JSON, modèle d’identité et échanges TNC2.